

Faculty of Computing

BSc. (Hons) in Computer Systems Engineering

Year 3 – Semester 1 – 2026 july

IE3064: Embedded Systems Engineering
Assignment - Smart Traffic Junction Controller

Student Name : R.A.H.S.Rajapaksha IT24610804

Student Name : R.M.G.C.Bandara IT24104321

Date of submitted : 2026 / 10 / 13

Table of Contents

1.ABSTRACT 5

1.1.Introduction 5
1.2.Problem and Motivation 6
1.3.Objectives : 6
1.4.Scope 6

2. REQUIREMENTS AND DESIGN DECISIONS 7

2. Key design decisions 7
2.2 Key design decisions 8

3. HARDWARE DESIGN 8

3.1.Component selection and Justification 9
3.2.Pin to Pin port mapping 10
3.3 Traffic light Module : LED current 11
3.4. Pin, port and Chip Current Check 11
3.5 Display, pedestrian LEDs, buzzer and button 12
3.6. LDR voltage devider and Threshold 13

4.FIRMWARE ARCHITECTURE 13

4.2. State Machine 14
4.4.Timer Configuration 15
4.5. Interrupt configuration 16
4.6.ADC configuration 16

5.SUB ROUTINE FLOWCHARTS 18

5.1. Phase cycle – UPDATE_TRAFFIC 18

5.2. PEDESTRIAN HANDLING 19

5.3.Night mode detection – UPDATE_NIGHT_MODE 21
5.4.countdown display – UPDATE_OUTPUT 22

6.IMPLEMENTATION AND TOOLCHAIN 23

6.1.Development steps 23
6.2. Source files 23
6.3. Build and upload commands 23
6.4. implementation notes worth highlighting 24

7 .TESTING AND RESULTS 25

7.1 Test method 25
7.2Normal mode = sequence and timing 25
7.3 Pedestrian crossing 26

7.4.Night mode and hysteresis 27
8. DISCUSSION 28
8.1 What worked well 28
8.2.Problems we faced and how we solved them 28
8.3 Limitations 29
8.4.Possible improvements 29
9. INDIVIDUAL CONTRIBUTION 30
10. DISCUSSION 31
11 CONCLUSION 31
12. REFERENCES 32
APPENDIX A – PHOTOGRAPHS OF THE COMPLETED MODEL 33
APPENDIX B – SCHEMATIC 37

Table of Figures

Figure 1 : State Machine 14
Figure 2 : Phase cycle – UPDATE_TRAFFIC 18
Figure 3 : ISR restarts a 30 ms timer 19
Figure 4 : Timer1 ISR counts it down 19
Figure 5 : READ_BUTTON checks 20
Figure 6 : Night mode detection – UPDATE_NIGHT_MODE 21
Figure 7: countdown display – UPDATE_OUTPUT 22
Figure 8 : assembly code build 24
Figure 9 : uploaded commands 24
Figure 10 : traffic light junction 33
Figure 12 : south approach 34
Figure 11 : pedestrian crossing 34
Figure 13 : Arduino uno 34
Figure 14 : Normal mode 35
Figure 15 : pedestrian with countdown 35
Figure 16 : Night mode 36
Figure 17 : schematic diagram 37

Tables

TABLE 1:KEY DECISION 7
TABLE 2: .COMPONENT SELECTION AND JUSTIFICATION 9
TABLE 3: PIN TO PIN PORT MAPPING 10
TABLE 4: MODULE CURRENT FLOW 11
TABLE 5: PIN, PORT AND CHIP CURRENT CHECK 11
TABLE 6: GROUP CURRENT 12
TABLE 7: PROGRAM STRUCTURE 14
TABLE 8 : STATE MACHINE 15
TABLE 9 : TIMER CONFIGURATION 15
TABLE 10 : INTERRUPT CONFIGURATION 16
TABLE 11 : ADC CONFIGURATION 16
TABLE 12 : SRAM VARIABLES 17
TABLE 13 : COUNTDOWN TABLE 23
TABLE 14 : SOURCE FILE 23
TABLE 15 : NORMAL MODE = SEQUENCE AND TIMING 25
TABLE 16 : NORMAL MODE = SEQUENCE AND TIMING 26
TABLE 17 : PEDESTRIAN CROSSING 27
TABLE 18 : NIGHT MODE AND HYSTERESIS 27
TABLE 19 : PROBLEMS WE FACED AND HOW WE SOLVED THEM 28
TABLE 20 : INDIVIDUAL CONTRIBUTION 30

1.Abstract

We designed and implemented a 60cm X 60cm model of a four-way junction using an ATmega328P 16MHz microcontroller (Arduino Uno) and only AVR Assembly. Normally the lights are controlled by a cycle that is programmed into a table in program memory, consisting of 10 s of green and 3 s of yellow and 1 s of all red per axis, followed by 1 s of all yellow (the lights are set to 'prepare' for green) before green is presented.

A flag is set by a pin-change interrupt on the South approach to the pedestrian button. The crossing is inserted only when the current phase and all-red clearance are over, and a 7-segment display counts down from 5, 4, 3, 2, 1, 0 at 1 s steps with all vehicle lights red. ADC channel 5 is connected to an LDR which toggles to Night mode and all 4 yellows flash 1Hz. The system only switches back to Normal mode when 5000 consecutive 1ms samples are above the threshold. All behaviour was tested in simulation and on hardware, and the results are presented in Section 7.

1.1.Introduction

A common example of an embedded system is traffic signals. They should always be on time, and they need to be able to hear and see the pedestrian at any time and adjust to the environment without failing. The present report describes the design of Smart Traffic Junction Controller for 4 way junction model for the module IE3064 in an ArduinoUno(ATmega328P, 16 MHz). All timers, interrupts, ADC reading, port writing etc. is done at register level and the firmware is written in AVR Assembly.

There are three modes for the controller:

- **Normal mode** is a cycling of North-South and East-West lights in green, yellow and all red, with timing set in a program memory table.
- **The 7-segment display counts down** from 5 to 0 and keeps all the lights in a pedestrian mode from the "all red" phase.
- **The Night mode** is based on an LDR to sense the darkness and flashes all the yellows at 1 Hz when the ambient light is less, and returns to normal after 5 seconds of continuous light.

1.2.Problem and Motivation

A junction controller needs to manage conflicting movements and to respond to the asynchronous pedestrian input without abruptly terminating a vehicle phase. The educational model is a combination of timing, input interrupts, analogue sensing and output masking used in a deterministic embedded application. It is not a certified road traffic controller; rather, it is a laboratory demonstration.

1.3.Objectives :

- Use complete application in AVR Assembly using direct register access.
- Generate the following NS/EW sequence and specify the duration of each program memory phase in the program memory.
- Respond to one pedestrian request and provide service after the proper clearance.
- Show the 5 second crossing countdown and then return the appropriate next vehicle phase. Use ADC light sensing for 1 Hz Yellow flashing and 5 second bright recovery.
- Document component choices, complete wiring, state transitions, tests and individual work.

1.4.Scope

The model is based on a South-side crossing and one countdown display. The crossing interval is indicated by extra pedestrian lamps and the use of an active buzzer. The current source does not implement a separate East countdown, a shift register, traffic-density sensing or independent per-approach green timing. Those features CAN NOT be claimed until the hardware and source is updated.

2. Requirements and design decisions

2. Key design decisions

Requirements	Implementation in current design
Twelve traffic LEDs and 4 approaches	Six PB0–PB5 outputs; each output drives two individually resisted LED branches.
Normal cycling with yellow and all-red clearance	10 s green, 3 s stopping yellow, 1 s clearance; additional 1 s ready yellow.
Request deferred until phase and clearance finish	v_request is checked at AFTER_NS or AFTER_EW expiry.
Five-second countdown from 5 to 0	PEDESTRIAN lasts 5000 ms; 0 appears in the following READY state.
Repeated presses produce one crossing	One pending flag; presses during PEDESTRIAN ignored.
Resume correct next phase	v_resume stores EW_READY or NS_READY.
All-yellow 1 Hz Night mode	500 ms ON plus 500 ms OFF, all other indicators off.
Five consecutive bright seconds for recovery	ADC > 400 increments recovery; ADC ≤ 400 resets it.
Document Night button policy	Requests in Night mode are ignored and a pending request is cleared on entry.
Model about 60 cm by 60 cm	Four labelled roads; South button/display; accessible LDR. Photograph and measure actual model.

Table 1:key decision

2.2 Key design decisions

• One 1 ms time base.

Timer1 provides a tick of 1ms, and each timer in the program keeps track of the number of ticks. This is the alternative to delay() and maintains responsiveness as there is no routine that blocks.

• Table-driven state machine.

The light patterns and durations are stored in flash tables, keyed by state number. Changing a timing involves changing one number, and the assessor can read all the phase timing in one location.

• Deferred, single-flag request.

The request is one byte, not a counter, and so ten presses will still give one crossing. The flag is only used for an all red and is only revealed at the end of an all-red. Any requests during the night are disregarded. We decided not to wait, and rather ignored, or a stored request could be a surprise crossing several minutes later when the user is gone. The flag will also be cleared when the night mode is turned on and off.

• Safe return from night.

When exiting night mode, it returns to AFTER_EW (all red), and then to NS_READY, and finally to NS_GO. Drivers do not change from flashing yellow to green without any hesitation.

• Get the "Prepare" (READY) states yellow before green.

It is similar to the red-amber signal in the UK and Sri Lanka, except that each axis flashes yellow for 1 s, followed by green. It alerts drivers to their green is coming, and gives the "0" of the countdown its own second.

3. Hardware Design

When designing ,the Hardware uses 18 of the Arduino Uno I/O pins :

- 6 pins – traffic lights
- 7 pins – seven segment display
- 2 pins – pedestrian indication LEDs
- 1 pin – buzzer
- 1 pin – push button
- 1 pin – LDR

3.1.Component selection and Justification

Component	Qty	Value / part	Why we chose it
Arduino Uno R3	1	ATmega328P	Required by the brief; 20 I/O pins, 10-bit ADC, Timer1 and pin-change interrupts
Traffic LED modules 	4	8 mm red, yellow and green LEDs with on-board 330 Ω, 220 Ω and 470 Ω resistors; pins G, Y, R, GND; active HIGH	One signal head per approach that looks like a real signal; resistors are built in, so wiring is short and neat
7-segment display	1	0.56 in, common anode	One digit is enough for 5 to 0; large digits are readable by the assessor
Segment resistors	7	330 Ω	One per segment so every digit is equally bright
Push button	1	12 mm tactile, momentary	Uses the internal pull-up, so no external resistor is needed
LDR + fixed resistor	1 + 1	GL5528 + 10 kΩ	Large dark/light change; 10 kΩ puts room light mid-scale on the ADC
Pedestrian LEDs	2	1 red, 1 green + 330 Ω each	Shows "wait" or "walk" on the South approach (optional extra allowed by the brief)
Buzzer	1	5 V active buzzer	Sound for visually impaired pedestrians during the crossing
power		+5	5v pin in Arduino Uno
Base board	1	60 cm × 60 cm board	Size set by the brief; roads marked at the centre

Table 2: .Component selection and Justification

3.2.Pin to Pin port mapping

Function	Uno pin	AVR signal	Direction and active level
North and South RED	D8	PB0	Output HIGH
North and South YELLOW	D9	PB1	Output HIGH
North and South GREEN	D10	PB2	Output HIGH
East and West RED	D11	PB3	Output HIGH
East and West YELLOW	D12	PB4	Output HIGH
East and West GREEN	D13	PB5	Output HIGH
Pedestrian push button	D7	PD7 / PCINT23	Input LOW; pull-up enabled
Pedestrian RED	A0	PC0	Output HIGH
Pedestrian GREEN	A1	PC1	Output HIGH
Display segment a	A3	PC3	Output LOW
Display segment b	A2	PC2	Output LOW
Display segment c	D2	PD2	Output LOW
Display segment d	D3	PD3	Output LOW
Display segment e	D4	PD4	Output LOW
Display segment f	D5	PD5	Output LOW
Display segment g	D6	PD6	Output LOW
Buzzer	A4	PC4	Output HIGH
LDR	A5	PC5 / ADC5	Analogue input; pull-up off

Table 3: Pin to Pin port mapping

3.3 Traffic light Module : LED current

Each module has its own resistor in series with each LED, so the Arduino pin only has to supply the LED current .

For one LED :

$$I_{LED} = \frac{V_{OH} - V_F}{R}$$

V_OH = HIGH voltage = 5V (Ideal) , 4.2V (loaded)

V_F = LED's forward voltage

The lowest HIGH voltage the ATmega328P datasheet guarantees when a pin supplies 20mA; it gives the dinnest LED. The real current lies between the two.

Colour	Resistor on module	V_F (typical)	Ideal, V_OH = 5.0 V	Loaded, V_OH = 4.2 V
Red	330 Ω	2.0 V	(5.0 − 2.0) / 330 = 9.1 mA	(4.2 − 2.0) / 330 = 6.7 mA
Yellow	220 Ω	2.0 V	(5.0 − 2.0) / 220 = 13.6 mA	(4.2 − 2.0) / 220 = 10.0 mA
Green	470 Ω	2.2 V	(5.0 − 2.2) / 470 = 6.0 mA	(4.2 − 2.2) / 470 = 4.3 mA

Table 4: module current flow

3.4. Pin, port and Chip Current Check

Each traffic pin drives the same colour on two modules wired in parallel , so the pin current is double the LED current:

$I_{pin} = 2 \times I_{LED}$

Pins	Signal	Ideal	Loaded	Check against the datasheet
PB0, PB3	NS / EW red	18.2 mA	13.4 mA	OK: under the 20 mA test condition
PB1, PB4	NS / EW yellow	27.3 mA	20.0 mA	Above 20 mA, but under the 40 mA absolute maximum
PB2, PB5	NS / EW green	11.9 mA	8.6 mA	OK: under 20 mA

Table 5: Pin, port and Chip Current Check

The only ones above 20 mA are the yellow ones, and they are still significantly below the absolute maximum of 40 mA. During Normal mode every yellow pin is on for 4 s out of 30 s (1 s READY + 3 s STOPPING) and flashes 50 % duty at night, which means that the average current is at or below 13.7 mA.

Port group. The datasheet restricts the total current for the supply group for PB0–PB5 to 100 mA. Table 3.5 sums up the LEDs that are on for each pattern from STATE_LIGHTS, assuming ideal (higher) values.

PORTB pattern	LEDs lit	Group current
0x0C, 0x21 (one axis green)	2 green + 2 red	11.9 + 18.2 = 30.1 mA
0x0A, 0x11 (one axis yellow)	2 yellow + 2 red	27.3 + 18.2 = 45.5 mA
0x09 (all red, crossing)	4 red	2 × 18.2 = 36.4 mA
0x12 (night, flash on)	4 yellow	2 × 27.3 = 54.6 mA

Table 6: group current

Whole chip - The ATmega328P has 200mA of current available on its VCC and GND connections. So 45.5mA (traffic) + 9.1mA (pedestrian red) + some 15mA (cpu) = about 70mA of current is flowing through the output pins into VCC (yellow phase). Current through the display is sunk through GND, the most current is 54.6 mA (digit 0) + 15 mA, about 70 mA. Both are 33.3% of the limit.

3.5 Display, pedestrian LEDs, buzzer and button

7-segment display. The display is common anode, meaning that when a segment pin is LOW current is being drawn:

$$I_{seg} = \frac{5.0 - V_F}{R}$$

$$= \frac{(5V - 2V)}{330\Omega}$$

$$\approx 9.1\text{mA}$$

The widest digit is 0, and a = f = 6 (9.1 mA each) → a total of 54.6 mA is consumed by the display at its widest setting. It is spread over three port groups: c, d, e on PD2–PD4 (27.3 mA), f on PD5 (9.1 mA) and a, b on PC3 and PC2 (18.2 mA), all far below each group's 100 mA limit.

Pedestrian LEDs (5 mm, 330 Ω each): red (5.0 − 2.0) / 330 = 9.1 mA, green (5.0 − 2.2) / 330 = 8.5 mA. Only one of them is illuminated at a time.

3.6. LDR voltage devider and Threshold

$$V_{A5} = 5 \times \frac{10\,k\Omega}{R_{LDR} + 10\,k\Omega},$$

$$ADC = \frac{V_{A5}}{5} \times 1024$$

The threshold of 400 counts equals .

R_{LDR}=15.6kΩ

Vin = 5 x (10/(15.6+10))

≈ 1.95 V

ADC =(1.95V /5) x 1024

which happens when the LDR reaches about 15.6 kΩ. Dark if reading is 400 or less. In room light covered the GL5528 is approximately 10 kΩ (about 512 counts) and covered it will rise above 100 kΩ (under 100 counts) so a hand is noticeable.

4.Firmware Architecture

4.1. Program Structure

Routine	**Type**	**Task**
RESET_HANDLER	Start-up	Sets the stack, clears SRAM variables, configures ports, ADC, Timer1 and PCINT, enters NS_GO
MAIN_LOOP	Loop	Waits for v_ticks > 0, takes one tick atomically, calls TICK_1MS
TICK_1MS	Scheduler	Night check, then button + traffic (Normal) or button sync (Night), then outputs
ISR_T1_COMPA	Interrupt	Every 1 ms: v_ticks++, counts the debounce timer down
ISR_PCINT2	Interrupt	Any edge on PD7: restarts the 30 ms debounce timer
READ_BUTTON	Subroutine	Accepts a stable press and sets v_request
READ_LDR	Subroutine	One ADC conversion on ADC5, result in r25:r24
UPDATE_NIGHT_MODE	Subroutine	Enters night on dark, counts 5000 ms of light to leave

UPDATE_TRAFFIC	Subroutine	Counts state time, chooses the next state, inserts the crossing
UPDATE_OUTPUTS	Subroutine	Writes PORTB lights, pedestrian LEDs, buzzer and 7-segment digit
ENTER_STATE / SYNC_BUTTON	Helpers	Change state and reset its timer / accept the current button level

Table 7: Program Structure

4.2. State Machine

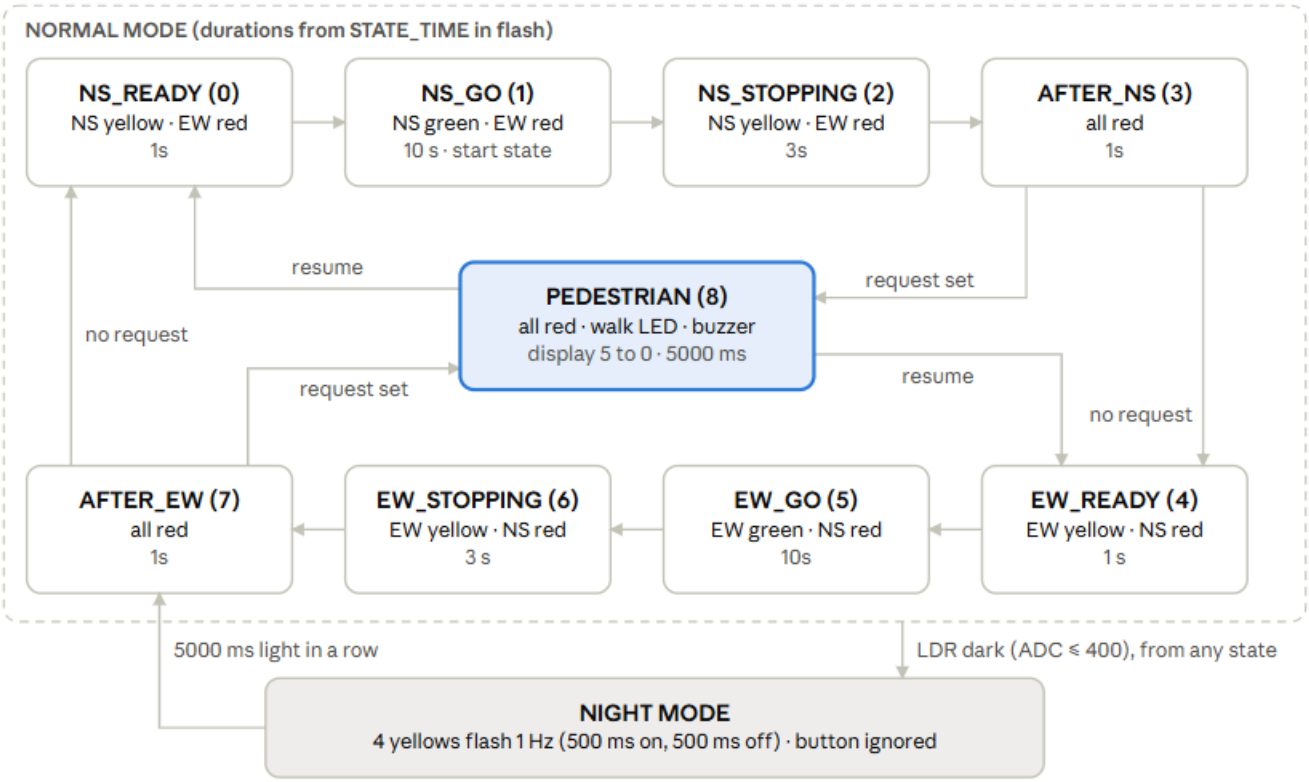


Figure 1 : State Machine

	State	NS lights	EW lights	PORTB pattern	Duration (ms)	Next state
0	NS_READY	Yellow	Red	0x0A	1000	NS_GO
1	NS_GO	Green	Red	0x0C	10000	NS_STOPPING
2	NS_STOPPING	Yellow	Red	0x0A	3000	AFTER_NS
3	AFTER_NS	Red	Red	0x09	1000	EW_READY, or PEDESTRIAN if requested
4	EW_READY	Red	Yellow	0x11	1000	EW_GO
5	EW_GO	Red	Green	0x21	10000	EW_STOPPING
6	EW_STOPPING	Red	Yellow	0x11	3000	AFTER_EW
7	AFTER_EW	Red	Red	0x09	1000	NS_READY, or PEDESTRIAN if requested
8	PEDESTRIAN	Red	Red	0x09	5000	Saved resume state (4 or 0)
—	NIGHT (mode)	Flash yellow	Flash yellow	0x12 / 0x00	500 on + 500 off	AFTER_EW after 5 s of light

Table 8 : State Machine

4.4.Timer Configuration

The timer runs in CTC mode, so it counts from 0 to OCR1A, raises an interrupt and restarts at 0.

$$f_{tick} = \frac{f_{CPU}}{N \times (1 + OCR1A)}$$

$$= \frac{16\,000\,000}{64 \times (249)} = 1000 \text{ Hz}$$

Register	Value	Meaning
TCCR1A	0x00	No PWM output, WGM11:10 = 00
TCCR1B	(1<<WGM12) | (1<<CS11) | (1<<CS10)	CTC mode 4, prescaler 64
OCR1AH:OCR1AL	0:249	250 counts = 1 ms (high byte written first)
TCNT1	0	Start counting from zero
TIMSK1	(1<<OCIE1A)	Enable compare-match A interrupt

Table 9 : Timer Configuration

4.5. Interrupt configuration

Item	Setting	Meaning
Vector OC1Aaddr	jmp ISR_T1_COMPA	Timer1 compare A, every 1 ms
Vector PCI2addr	jmp ISR_PCINT2	Pin change on PORTD
PCMSK2	(1<<PCINT23)	Only PD7 can trigger it
PCIFR	(1<<PCIF2)	Clears any old flag before enabling (write 1 to clear)
PCICR	(1<<PCIE2)	Enables the PORTD pin-change group
SREG I-bit	sei after setup	Global interrupt enable

Table 10 : Interrupt configuration

4.6.ADC configuration

Register	Value	Meaning
ADMUX	(1<<REFS0) | 5	AVcc (5 V) reference, right-adjusted, channel ADC5
ADCSRA	(1<<ADEN) | (1<<ADPS2:0)	ADC on, clock 16 MHz / 128 = 125 kHz
DIDR0	(1<<ADC5D)	Digital input buffer off on PC5 to save power and noise

Table 11 : ADC configuration

4.7. SRAM Variables

Variable	Size	Purpose
v_ticks	1 B	Pending 1 ms ticks (written by Timer1 ISR)
v_debounce	1 B	Debounce countdown in ms
v_btnCheck	1 B	1 = button quiet for 30 ms, check it
v_stable	1 B	Last accepted button level
v_state	1 B	Current state 0–8
v_resume	1 B	State to resume after the crossing
v_request	1 B	Pedestrian request flag
v_nightMode	1 B	1 = Night mode
v_showZero	1 B	Show "0" in the READY state after a crossing
v_stateMs	2 B	ms spent in the current state
v_nightMs	2 B	Flash timer 0–999 ms
v_recovMs	2 B	Continuous light time in ms
v_ldr	2 B	Last ADC reading

Table 12 : SRAM Variables

5.Sub routine Flowcharts

The four routines the brief names each have one flow chart :

5.1. Phase cycle – UPDATE_TRAFFIC

In normal mode there is a call once per millisecond . It adds 1 to v_stateMs, reads this state's duration from STATE_TIME with LPM and , when time is up , picks the next state. The pedestrian flag is checked only at the end of the All red state.

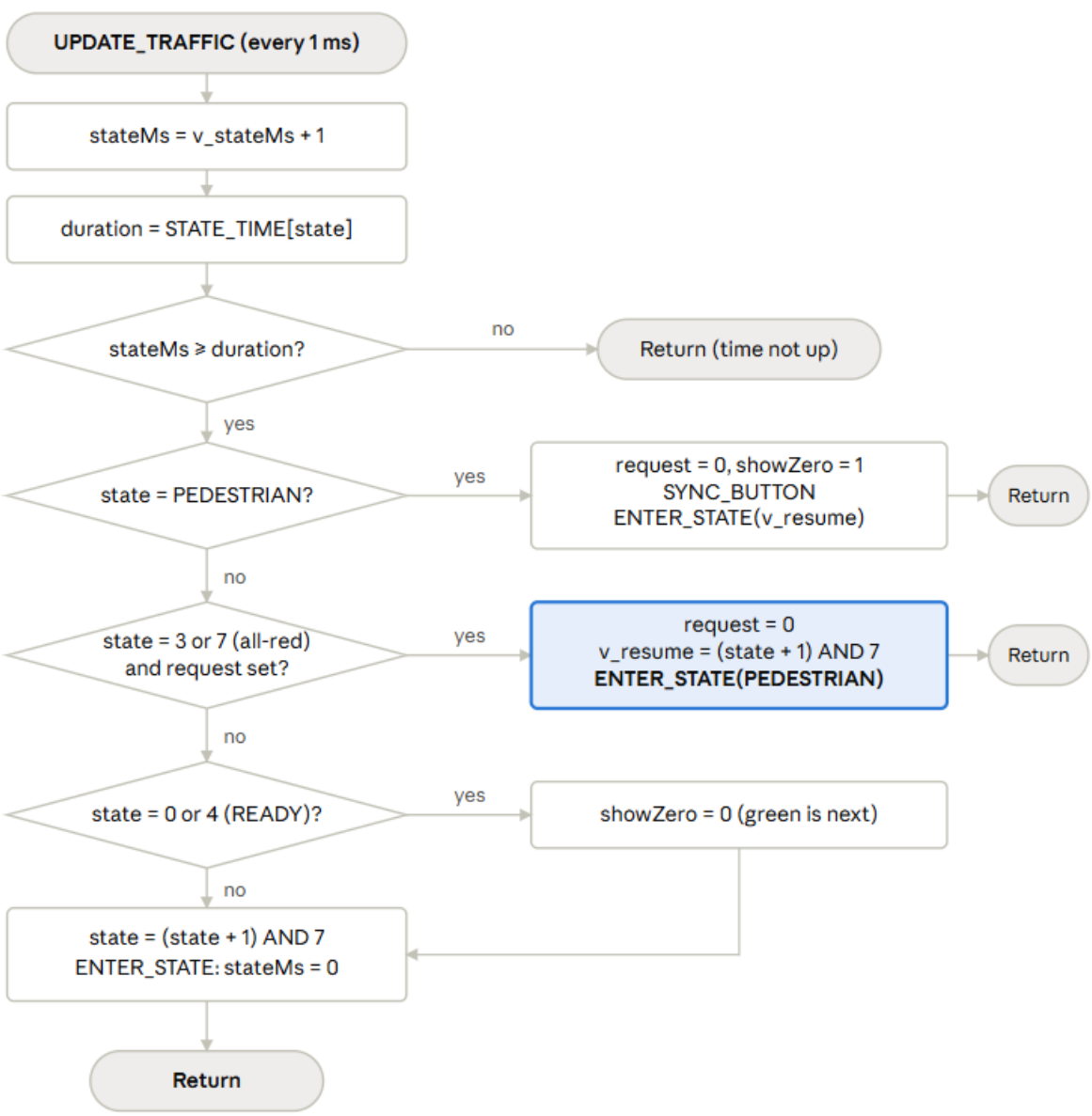


Figure 2 : Phase cycle – UPDATE_TRAFFIC

The AND 7 wraps state 7 back to 0, so the 8th state repeats forever; the crossing saves the next state in v_resume and return to it.

5.2. pedestrian handling

A press handled in 3 small steps :

1. The pin-change ISR restarts a 30 ms timer on every bounce

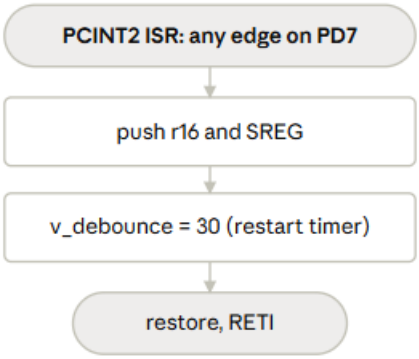


Figure 3 : ISR restarts a 30 ms timer

2. Timer1 ISR counts it down

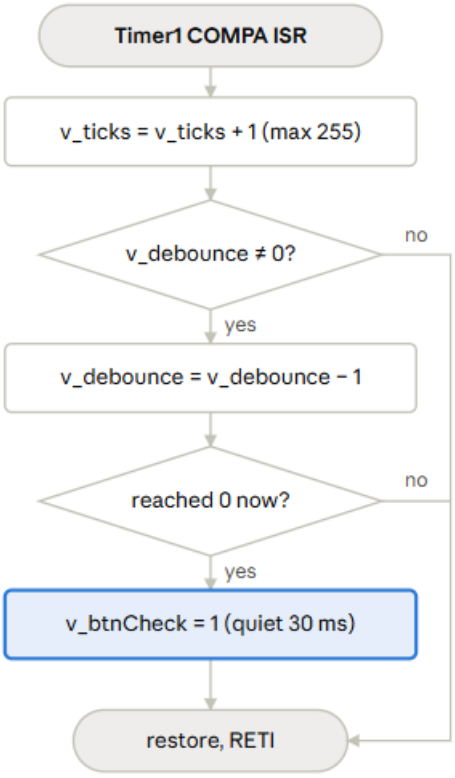


Figure 4 : Timer1 ISR counts it down

3. READ_BUTTON checks the pin only after 30 ms of silence.

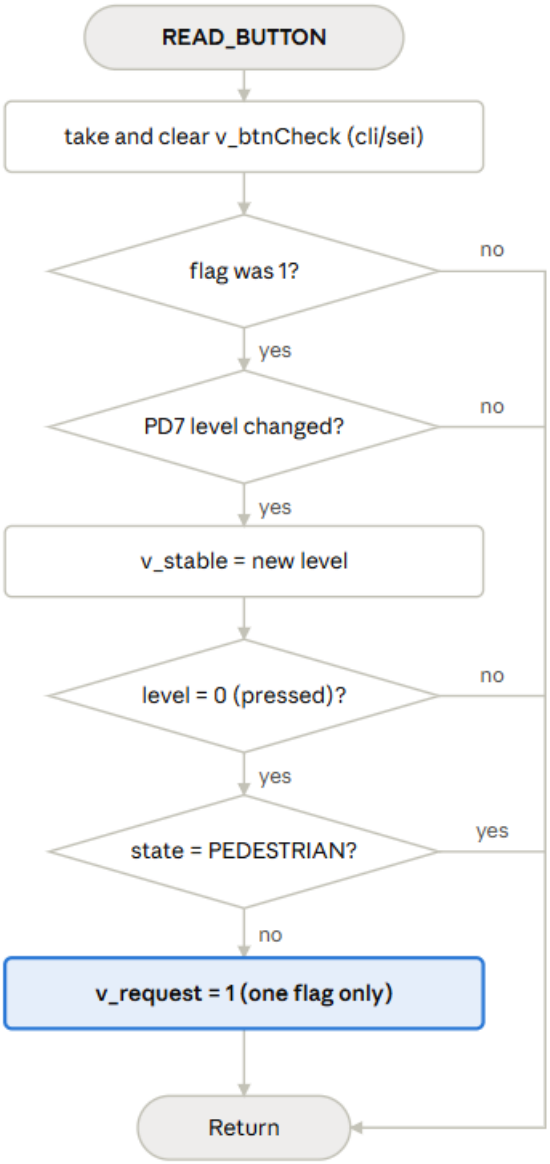


Figure 5 : READ_BUTTON checks

The 3 parts talks only through 2-bytes :

- Lane 1 wires v_debounce
- Lane 2 wires v_btnCheck after 30 quite milliseconds
- Lane 3 turns that into v_request.

5.3.Night mode detection – UPDATE_NIGHT_MODE

Runs every 1ms both modes . one dark sample enters night mode at once and resets the recovery counter. Leaving needs 5000 light samples in a row, so a quick uncover and recover keeps Night mode.

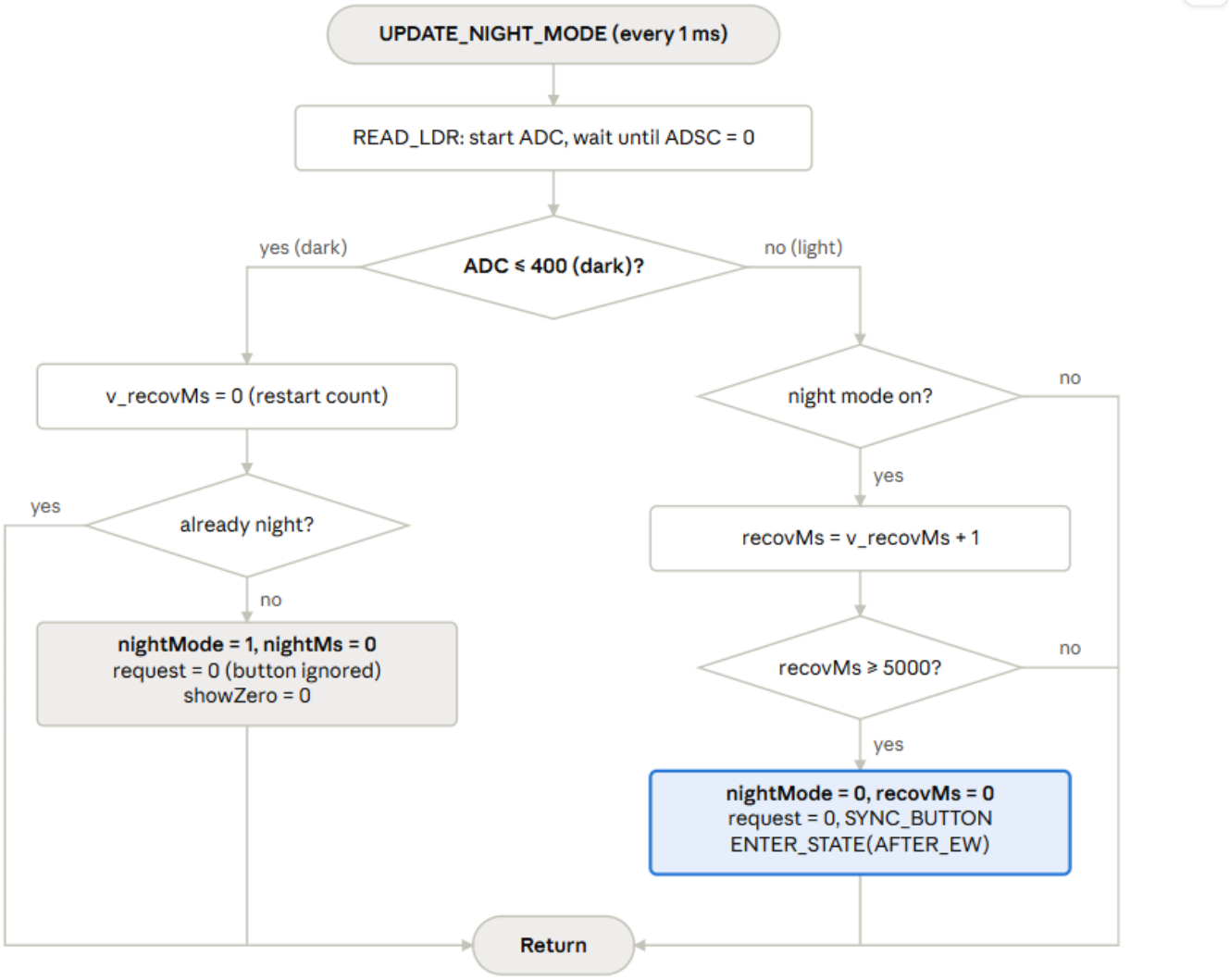


Figure 6 : Night mode detection – UPDATE_NIGHT_MODE

The routine runs 1000 times a second, so 5000 light samples in a row = exactly 5s. leaving goes through AFTER_EW, giving 1s of all- red before any green.

5.4.countdown display – UPDATE_OUTPUT

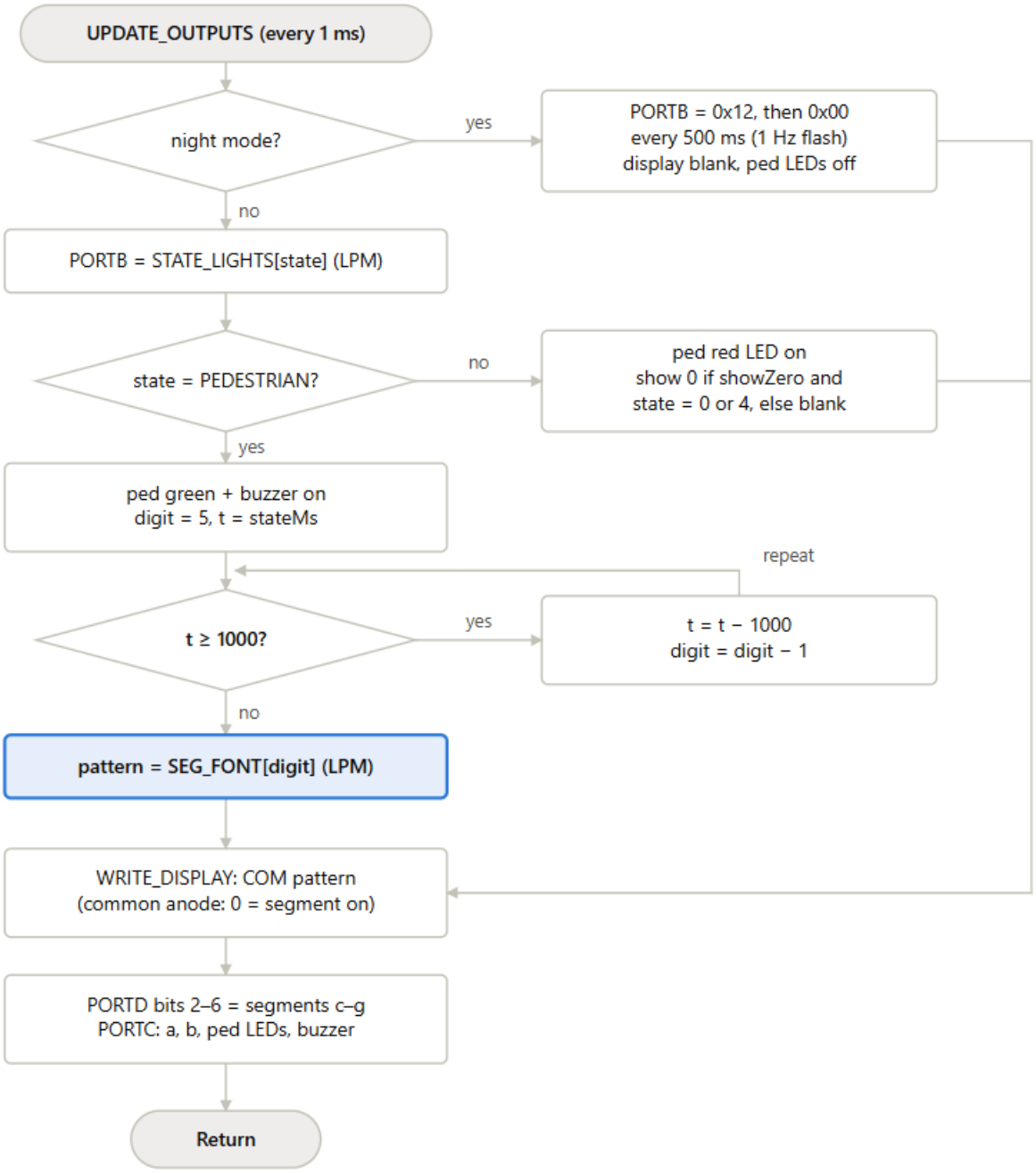


Figure 7: countdown display – UPDATE_OUTPUT

Time in PEDESTRIAN (ms)	Digit shown	Segments lit (SEG_FONT)
0–999	5	0x6D: a, c, d, f, g
1000–1999	4	0x66: b, c, f, g
2000–2999	3	0x4F: a, b, c, d, g
3000–3999	2	0x5B: a, b, d, e, g
4000–4999	1	0x06: b, c
next 1000 ms (READY state)	0	0x3F: a, b, c, d, e, f

Table 13 : countdown table

6.Implementation and toolchain

The same applies to the two dialects of assembler used: traffic.asm for avr-as (which is required for submission by the brief), and main.asm for Microchip Studio (which we used to simulate and debug). Both give the same behaviour in simulation.

6.1.Development steps

- To finalize the timing, the order of the states and the threshold for the LDR was quickly done by **prototyping in Arduino C++** (ADC readings were printed on the Serial port). This is a prototype that is not submitted.
- **Hand translate to AVR assembly,** with each library call replaced with registers: millis() -> Timer1 ISR, digitalRead -> PCINT + debounce, analogRead -> ADC registers, digitalWrite -> whole-port writes.
- **Simulate** in Microchip Studio and simavr, checking every timing
- **Upload and test on hardware ,** then record results and photos.

6.2. Source files

File	Assembler	Purpose
main.asm	GNU avr-as (AVR-GCC)	Submitted firmware; assembles with no errors on a clean install
microchip_studio/main.asm	AVRASM2 (Microchip Studio 7)	Same logic for simulation and debugging
README.md	—	Toolchain setup, build and upload commands, file descriptions
traffic.hex	—	Ready-to-flash output

Table 14 : source file

6.3. Build and upload commands



Figure 8 : assembly code build

```
C:\Users\Asus>"%LOCALAPPDATA%\Arduino15\packages\arduino\tools\avrdude\8.0.0-arduino1\
bin\avrdude.exe" -c arduino -p atmega328p -P COM8 -b 115200 -U flash:w:"C:\Users\Asus\
Downloads\AssemblerApplication2\AssemblerApplication2\Debug\AssemblerApplication2.hex"
:i
Reading 826 bytes for flash from input file AssemblerApplication2.hex
Writing 826 bytes to flash
Writing | ################################################## | 100% 0.21 s
Reading | ################################################## | 100% 0.13 s
826 bytes of flash verified
```

Figure 9 : uploaded commands

6.4. implementation notes worth highlighting

The following are some of the implementation notes of this claim that deserve mention.

• Atomic port writes. No half-pattern is ever seen as all six traffic signals are on PORTB, and one OUT changes all the signals.

• 16-bit maths on an 8-bit CPU. Timers use pairs of r25:r24 with ADIW and compare use CP then CPC.

• Flash tables. LPM reads bytes so the label of a table is multiplied by two to get its byte address. The .db lines of AVRASM2 are filled to an even number of bytes, so that SEG_FONT is kept at one line, and STATE_LIGHTS has one padding byte.

• Read-modify-write on PORTD. All the bits from 2-6 are only modified, so the PD7 pull-up remains active during the segment updates.

• Common-anode display. If the pin of a segment is 0, the segment will be lit, and the font pattern will be inverted before it is written.

7 .Testing and results

For each marking criterion there is a test in simulation and then the built model. All tests were passed in exact millisecond timings in simulation.

7.1 Test method

The simulation: simavr kept track of all PORTB, PORTC and PORTD changes and their time stamps. The test inputs to the button and LDR included deliberate contact bounce.

Hardware: Each test has been repeated 3 times. Phase times were determined from a video taken with the phone at 30 frames per second (fps) ± 0.03 s.

7.2Normal mode = sequence and timing

ID	Phase	Expected (s)	Simulation (s)	Hardware run 1 (s)	Run 2 (s)	Run 3 (s)	Result
N1	NS green (NS_GO)	10.0	10.000	10	10	10	Pass
N2	NS yellow (NS_STOPPING)	3.0	3.000	3	3	3	pass
N3	All-red (AFTER_NS)	1.0	1.000	1	1	1	Pass
N4	EW prepare yellow (EW_READY)	1.0	1.000	1	1	1	pass
N5	EW green (EW_GO)	10.0	10.000	10	10	10	Pass
N6	EW yellow (EW_STOPPING)	3.0	3.000	3	3	3	pass
N7	All-red (AFTER_EW)	1.0	1.000	1	1	1	Pass
N8	NS prepare yellow (NS_READY)	1.0	1.000	1	1	1	Pass
N9	Full cycle	30.0	30.000	30	30	30	pass

Table 15 : Normal mode = sequence and timing

ID	Check	Expected	Simulation	Hardware
N10	Order over 5 cycles	1→2→3→4→5→6→7→0→1, no skip or repeat	Pass	Pass
N11	All-red between every direction change	Present after NS yellow and after EW yellow	Pass	pass
N12	Never green on both axes	No pattern with both green bits set	Pass	pass
N13	Pedestrian red during vehicle phases	PC0 on, PC1 off	Pass	pass

Table 16 : Normal mode = sequence and timing

7.3 Pedestrian crossing

ID	Test	Expected	Simulation	Hardware
P1	Press 2 s into NS green	Green finishes its 10 s, then yellow 3 s, all-red 1 s, then crossing	Pass	pass
P2	Press during EW yellow	Yellow finishes, all-red 1 s, then crossing	Pass	Pass
P3	Lights at the moment of press	No light changes	Pass	pass
P4	All four directions during crossing	All red (PORTB = 0x09) for 5 s	Pass	pass
P5	Countdown	5, 4, 3, 2, 1 at 1 s each, then 0 for 1 s	Pass	pass
P6	Pedestrian LEDs and buzzer	Green + buzzer during crossing, red otherwise	Pass	pass
P7	5 quick presses with bounce	Exactly 1 crossing	Pass	pass
P8	Press during crossing	Ignored; no second crossing queued	Pass	pass
P9	Resume after crossing from AFTER_NS	Continues at EW_READY, then EW green	Pass	pass

P10	Resume after crossing from AFTER_EW	Continues at NS_READY, then NS green	Pass	pass
P11	Press right after a crossing	Next crossing only after one full vehicle green	[RUN in sim]	pass

Table 17 : Pedestrian crossing

7.4.Night mode and hysteresis

ID	**Test**	**Expected**	**Simulation**	**Hardware**
M1	Cover LDR	Night mode within one sample (1 ms)	Pass	Pass
M2	Flash rate	All 4 yellows 0.5 s on, 0.5 s off (1 Hz); all other LEDs and display off	Pass	pass
M3	Uncover for 5 s	Stays in night until 5.000 s of light, then all-red 1 s → NS yellow 1 s → NS green	Pass	pass
M4	Uncover 3 s, re-cover, uncover	Counter resets; night lasts 5 s after the final uncover	Pass	pass
M5	Press button at night	Ignored, no crossing after return to Normal	Pass	Pass
M6	Cover during a crossing	Night mode immediately; crossing cancelled	[RUN in sim]	Pass

Table 18 : Night mode and hysteresis

8. Discussion

All the design features were functional, and the main lesson was that it was easy to combine all the features with a single non-blocking 1 ms time base. A delay loop version was not able to monitor the button, LDR and flash timer simultaneously.

8.1 What worked well

• Accurate timing. The phase times matched to the millisecond in the simulation, Timer1 in CTC mode was giving exactly 1000 ticks per second from the 16 MHz crystal.

• Safe by construction. The request will be tested only at the end of an all-red state, thus the code does not have a way to go directly from green or yellow to a crossing.

• Clean debouncing. A 30 ms quiet timer added to the pin-change interrupt eliminated the busy-waiting and allowed for the one press of the button to be interpreted as exactly one request.

• Easy to change. All timings are stored in a single flash table and a new timings plan is simply a one-line edit.

8.2.Problems we faced and how we solved them

Problem	Cause for the problem	How we fixed
One press gave several requests	Contact bounce for a few ms	30ms restartable debounce timer (PCINT + Timer)
Wrong 7 segment digits at first	AVRASM pads each .db line to an even byte count, shifting the table	Kept SEG_FONT on one line
Table reads returned wrong bytes	LPM uses byte addresses , labels are word addresses	Multiply table labels by 2
Button pull-up switched off while updating segments	Write the whole PORTD byte cleared PD7	Masked write with ANDI 0 x83 to keep PD0, PD1, PD7
Night mode flickered near the threshold	LDR reading hovers around 400 in a dim room	Exit needs 5000ms of continuous light; threshold calibrated with the C++ prototype
Course requires avr-as, we debugged in Microchip Studio	Two assembler doalects	Kept two equivalent files and compared simulation traces

Table 19 : Problems we faced and how we solved them

8.3 Limitations

- The model cannot provide individual approaches a protected right turn or green signal, as this is shared by both North and South (and East and West).
- Night Mode is activated when a dark sample is taken. A 1ms shadow could switch modes, so a short entry delay would circumvent this.
- If the LDR is covered while crossing then the crossing will be cancelled. This is safe, since all lights turn to flashing yellow and normal mode restarts from all-red, but if a pedestrian pressed the lights would have to be pressed again.
- The "prepare" yellow before green is not a part of the sequence in the brief and some assessors may interpret it as an additional yellow. We will explain it in section 2.1.
- Times are determined at assembly. Each of the yellow pins has a current of 20–27 mA, as they power two module LEDs with 220 Ω resistors. This is higher than the 20 mA test condition, but less than the 40 mA absolute max; in a real product, each signal would go through .

8.4.Possible improvements

- A separate pin (or a 74HC595 shift register) for each approach for protected turns.
- Sleep mode (SLEEP in idle) between ticks to save battery.
- Store a timing plan in the EEPROM which can be edited without reflashing
- . A brief entry delay (e.g. 1 s of darkness) to make Night mode impervious to any transient shadows.
- Vehicle detection to extend green when heavy traffic (adaptive timing).

9. Individual Contribution

Area	Member 1: R.A.H.S.Rajapaksha	Member 2: R.M.G.C.Bandara
Firmware modules	Timer1 setup, MAIN_LOOP, UPDATE_TRAFFIC, state tables, READ_LDR, UPDATE_NIGHT_MODE, night flash	PCINT setup, debounce, READ_BUTTON, pedestrian insertion and resume, SEG_FONT, countdown and buzzer in UPDATE_OUTPUTS
Hardware	traffic light modules, LDR divider, power	Button, 7-segment display, pedestrian LEDs, buzzer driver, power
Report	Sections 3.3–3.5, 4, 5.1, 5.3	Sections 3.1–3.2, 5.2, 5.4, 7

Table 20 : Individual Contribution

Member 1 statement. Designed and wrote the traffic-light cycling and Night mode by me **[R.A.H.S.Rajapaksha , IT24610804**]. I set up Timer1 in CTC mode at the 1 ms tick, programmed the phase tables, programmed the ADC routine and programmed the 5 s hysteresis logic, and wired in the 12 traffic LEDs and the LDR divider. I tried the full test of all the phase times and night-mode cases in Section 7.

Member 2 statement. I [**R.M.G.C.Bandara, IT24104321**] designed and wrote the pedestrian crossing and countdown display. Set up the pin-change interrupt on PD7, programmed the 30ms debounce, programmed the deferred request logic, programmed the 7-segment font table and programmed the countdown routine; wired the button, the display, the pedestrian LEDs, and the buzzer. I have tried all the footway cases in Section 7.

10. Discussion

The finished controller was able to fulfill all the functional requirements of the brief as both a simulation and on the physical model. Three hardware runs were performed with all Normal-mode phase times matching the STATE_TIME table and all red phases between changes of direction, as well as no matching of the two green groups. The lights were never seen to change when pressed in Pedestrian mode, and five consecutive presses resulted in one crossing, with sufficient time in between for traffic to be resumed on the correct next axis. Under Night mode, the yellows were flashing at 1 Hz and re-covering the LDR resets the five second recovery counter as required.

The design decision was that the entire program should be based on a single (1ms) non-blocking Timer1 time base. These are the ticks that are counted by every timer in the system: phase duration, debounce, flash interval and night recovery. Since there is no delay loop in any of the routines, the controller may be able to monitor the button, read the LDR, and flash the lights simultaneously.

The structure of the state machine is what provides safety. A pedestrian request is just a flag (one byte) and is only examined during an all red state ending, so there is no path from a green or yellow light to a crossing. We also excluded button presses during night time, as a request which was queued for minutes might begin an unexpected crossing when the pedestrian was away.

The primary learning outcome was to work at the register level. LPM byte addressing, assembler padding of .db lines and the button pull-up being cleared by a whole-port write (Section 8.2) are problems that only occur without a library. We gained a clear understanding of the timers, interrupts, ADC and I/O ports of the ATmega328P by finding solutions to them.

11 Conclusion

We programmed and verified a Smart Traffic Junction Controller (STJC) using solely AVR assembly for ATmega328P microcontroller, and all three modes are functioning as expected:

- Normal mode is a 30 second cycle of yellow and all red clearance.
- Pedestrian mode takes turns, keeps all lights red for the 5-to-0 countdown, allows one crossing per request and restarts at the correct phase.
- The yellows will flash at 1 Hz and only return after 5 seconds of light in night mode.

An 1 ms Timer1 time base and a table driven state machine were sufficient to keep the system responsive, accurate and easily re-timed. The test results validated the correct behaviour in both simulation and hardware. The project allowed us to gain hands on experience with timers, interrupts, ADC, and setting up the port control at register level. Independent approach and timing plans stored in EEPROM could be added in future work.

12. References

[1] Microchip Technology Inc., *ATmega48A/PA/88A/PA/168A/PA/328/P Megaavr Data Sheet*, DS40002061B. Available: https://www.microchip.com/en-us/product/atmega328p

[2] Microchip Technology Inc., *AVR Instruction Set Manual*, DS40002198A. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/AVR-Instruction-Set-Manual-DS40002198A.pdf

[3] Microchip Technology Inc., *AVR Assembler User Guide*, DS40001917A. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/40001917A.pdf

[4] Microchip Technology Inc., "Microchip Studio for AVR and SAM Devices." Available: https://www.microchip.com/en-us/tools-resources/develop/microchip-studio

[5] Arduino, "UNO R3 – Documentation, pinout and schematics." Available: https://docs.arduino.cc/hardware/uno-rev3/

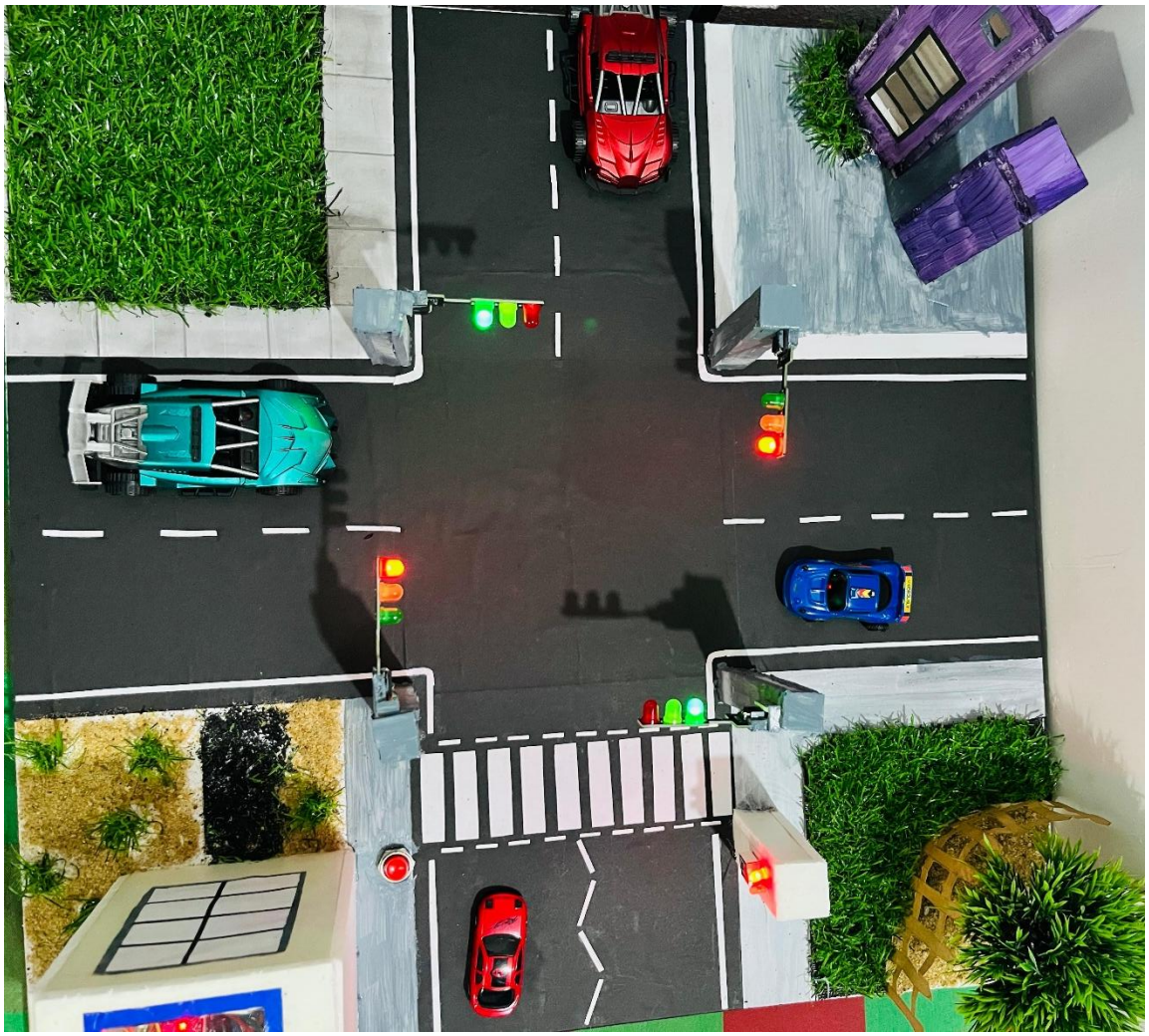
[6] ProtoSupplies, "Traffic Light LED Module (MOD-63)." Available: https://protosupplies.com/product/traffic-light-led-module/

[7] onsemi, *BC546B, BC547A/B/C, BC548B/C Amplifier Transistors NPN Silicon*, BC546/D, Rev. 8, Aug. 2021. Available: https://www.onsemi.com/pdf/datasheet/bc546-d.pdf

Appendix A – Photographs of the Completed model

Top view of the whole board

North



East

west

South

Figure 10 : traffic light junction

Close-up of the South approach: button, 7-segment display, pedestrian LEDs and buzzer



Figure 12 : south approach

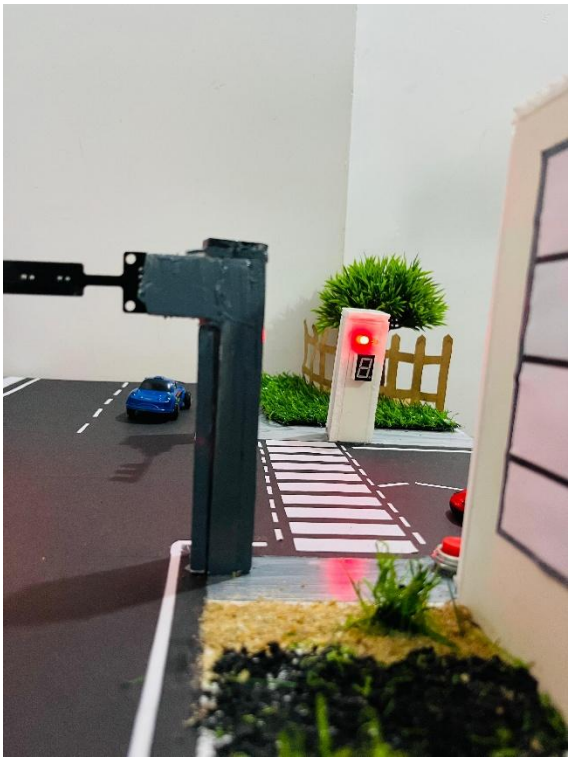


Figure 11 : pedestrian crossing

Arduino Uno

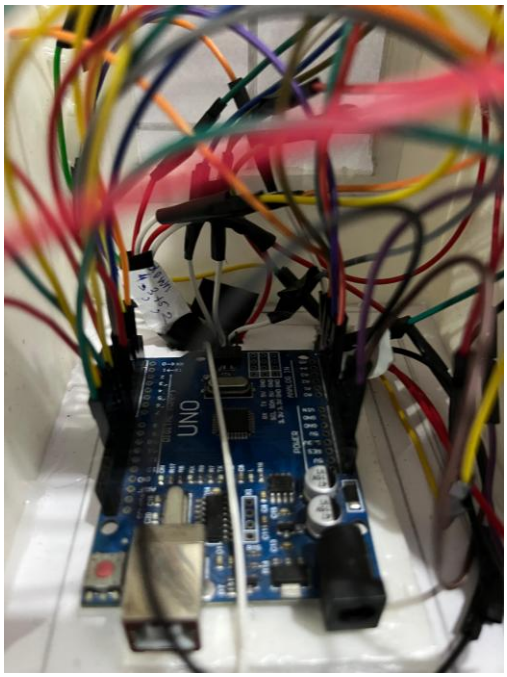


Figure 13 : Arduino uno

Normal mode (NS green)



Figure 14 : Normal mode

Pedestrian mode (all red, display showing a digit)



Figure 15 : pedestrian with countdown

Night mode (yellows lit) side by side

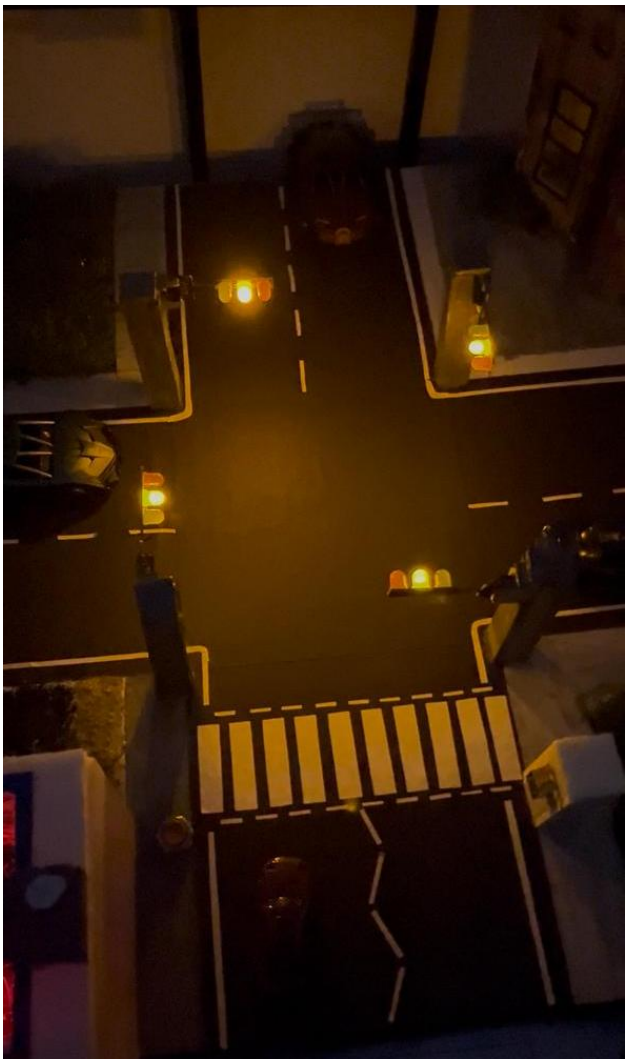


Figure 16 : Night mode

Appendix B – Schematic

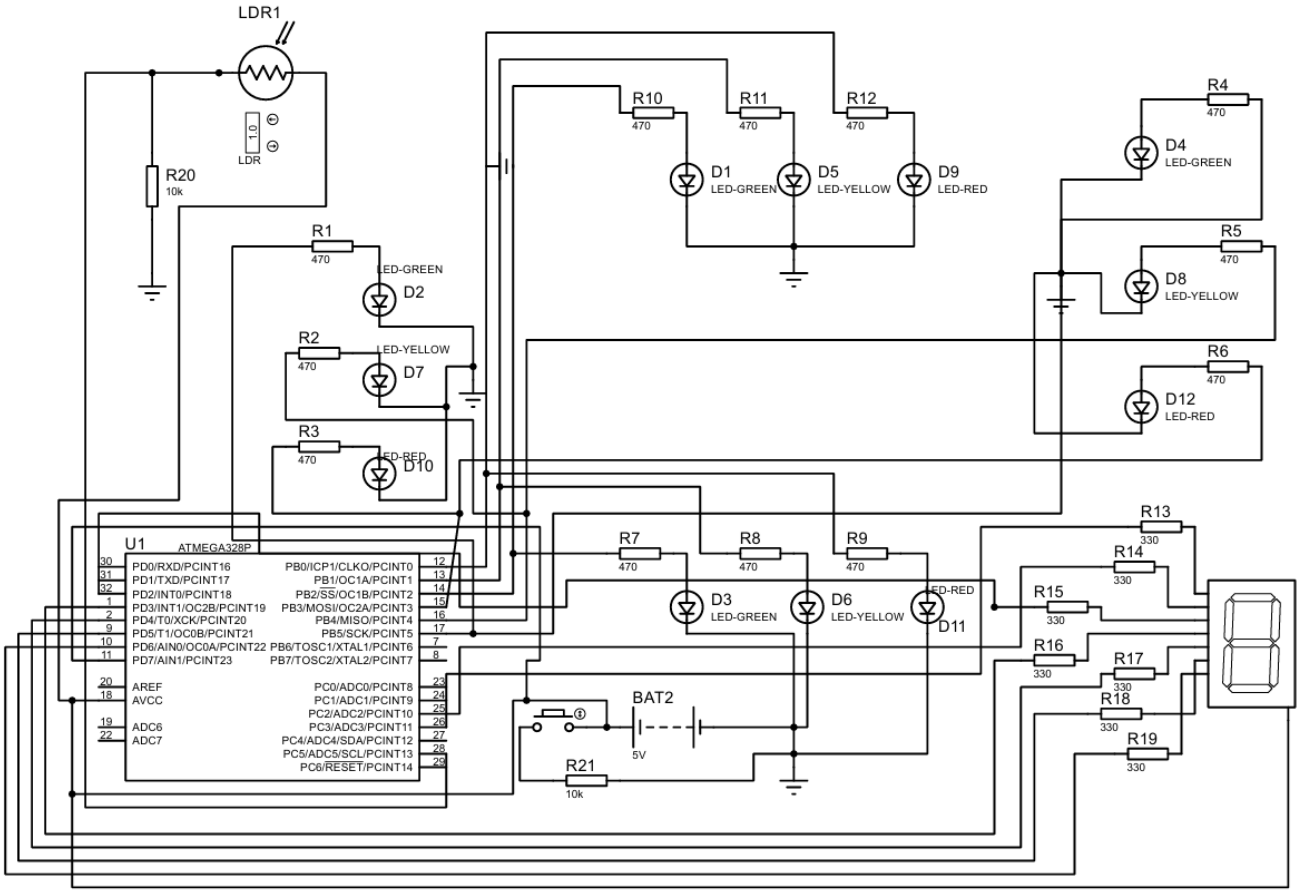


Figure 17 : schematic diagram